

5.2 - Consumer-driven contracts: testando compatibilidade entre serviços

Quando o consumidor define o que o provedor precisa entregar

O problema dos contratos implícitos entre serviços

Por que testes unitários e de integração não capturam tudo

- **Microserviços comunicam-se por contratos que ninguém formalizou explicitamente**
 - Schema do JSON, semântica dos campos, ordem de eventos, tudo é contrato implícito
- **Teste unitário valida o provedor em isolamento, sem conhecer o consumidor real**
- **Teste de integração ponta a ponta é caro e frágil em microserviços**
 - Cada serviço novo no fluxo multiplica a complexidade dos testes
- **Quebras silenciosas acontecem em produção, dias após o deploy do provedor**
 - O log do erro aparece no consumidor, longe da origem da mudança

Consumer-driven contracts, a inversão

Quem consome declara, quem provê valida

- **O consumidor escreve um contrato declarando exatamente o que espera receber**
 - Estrutura do JSON, tipos dos campos, status HTTP, headers relevantes
- **Esse contrato é publicado em um broker compartilhado entre os times**
 - Pact Broker é a ferramenta de referência no ecossistema
- **O provedor executa todos os contratos publicados como testes no seu pipeline**
 - Se algum contrato falha, o pipeline do provedor barra a mudança antes do merge
- **A responsabilidade pelo contrato passa do provedor para o consumidor**
 - Isso reflete a realidade, quem usa sabe melhor o que precisa que quem fornece

A sexta-feira em que a LunaLog perdeu uma semana de faturamento

- **Time de rotas refatora endpoint de cotação, distância retornada agora em quilômetros**
 - Time de billing depende desse campo para calcular o valor da entrega

LUNALOG

O time de rotas decidiu que retornar distância em metros era inconveniente, valores enormes em telas, parsing chato no front. Trocaram a unidade para quilômetros, deploy na sexta à tarde. O time de billing nunca foi avisado, ninguém olhou as integrações downstream. Na segunda-feira de manhã, o relatório financeiro mostrou que toda a semana havia sido faturada com valores mil vezes menores que o esperado. Reverter foi impossível, os boletos já estavam emitidos. Roberto descobriu naquela tarde o que era consumer-driven contracts. O time de billing, se tivesse declarado o contrato no broker, teria barrado o deploy de rotas automaticamente.

Provider-first contra consumer-first

Provider-first (modelo tradicional)

- Provedor publica documentação OpenAPI ou Swagger
- Consumidor lê e implementa contra a doc
- Quando provedor muda, doc é atualizada, ninguém é avisado
- Quebra acontece em runtime, no consumidor
- Confiança baseada em boa vontade, não em mecanismo

Consumer-first (CDC)

- Consumidor declara contrato como código de teste
- Contrato vira artefato versionado no broker
- Pipeline do provedor executa todos os contratos publicados
- Quebra é capturada antes do merge, no provedor
- Confiança garantida por automação, não por comunicação

CDC integrado ao pipeline, não como afterthought

- **Contract testing só funciona se for gate obrigatório de promoção**
- **Contrato sem execução no pipeline é apenas documentação melhor**

O ganho real de CDC vem da automação no pipeline do provedor. Cada pull request que altera o serviço executa os contratos publicados no broker. Se algum contrato quebra, o merge é bloqueado. Sem essa amarração, contract testing vira apenas mais uma camada de testes ignorada pelo time.



Contratos entre serviços validados, quebras detectadas antes do merge. Mas o pipeline que integra tudo isso, como ele deveria ser estruturado para dar confiança real, e não apenas automatizar o caos? Existe uma diferença entre integrar continuamente, entregar continuamente e implantar continuamente, e essa diferença muda completamente o que cada time precisa entregar.

Fundamentos de CI/CD, integração, entrega e implantação contínuas